

Minería de Texto Avanzada

</> Código ▼

AUTOR/A

Almacenes y Minería de Datos

MINERÍA DE TEXTOS

Minería de Texto Avanzada

TF-IDF: ponderar términos para representar documentos

1 Introduccion a la Minería de Texto Avanzada

[Omar]

2 TF-IDF

2.1 Planteamiento del problema

En minería de textos, los documentos no pueden analizarse directamente como cadenas de caracteres. Un algoritmo de clasificación, agrupamiento o similitud necesita una representación numérica. TF-IDF (*Term Frequency - Inverse Document Frequency*) resuelve este paso convirtiendo cada documento en un vector donde cada componente representa el peso de un término.

La idea central es distinguir entre dos tipos de palabras:

- Palabras frecuentes pero poco informativas: por ejemplo, “el”, “de”, “para”, “en”.
- Palabras frecuentes dentro de un documento y menos comunes en el corpus: por ejemplo, “filas”, “ductos”, “muro fronterizo”, “investigación”.

TF-IDF no asigna temas directamente. Lo que genera es una matriz de ponderaciones. La interpretación temática se obtiene después, observando qué términos tienen mayor peso en cada documento.

2.2 Representación vectorial del texto

Supongamos un corpus con N documentos:

$$\mathcal{D} = \{d_1, d_2, \dots, d_N\}$$

Después de limpiar y tokenizar los textos, se construye un vocabulario:

$$V = \{t_1, t_2, \dots, t_m\}$$

Cada documento se representa como un vector de longitud m :

$$d_i = (w_{i1}, w_{i2}, \dots, w_{im})$$

donde w_{ij} es el peso TF-IDF del término t_j en el documento d_i . Si un término no aparece en un documento, su peso es cero.

Filas

Cada fila de la matriz representa un documento del CSV.

Columnas

Cada columna representa una palabra o frase del vocabulario.

Celdas

Cada celda contiene el peso TF-IDF de un término en un documento.

2.3 Frecuencia del término

La frecuencia del término, o TF, mide la importancia local de una palabra dentro de un documento.

$$TF(t, d) = \frac{f(t, d)}{\sum_{t' \in d} f(t', d)}$$

Aquí $f(t, d)$ es el número de veces que el término t aparece en el documento d .

Una palabra repetida varias veces dentro del mismo documento obtiene un TF mayor. Por eso en el ejemplo, términos como **filas** en el documento 2 o **discurso** en el documento 6 empiezan a destacar frente a otros términos que solo aparecen una vez.

2.4 Frecuencia inversa en documentos

El IDF mide la especificidad global de un término dentro del corpus. Penaliza términos que aparecen en muchos documentos y favorece términos que aparecen en pocos.

$$IDF(t) = \log \left(\frac{N}{DF(t)} \right)$$

Donde:

- N es el número total de documentos.
- $DF(t)$ es el número de documentos donde aparece el término t .

En la implementación de `scikit-learn` se usa suavizado:

$$IDF(t) = \log \left(\frac{1 + N}{1 + DF(t)} \right) + 1$$

El suavizado evita divisiones problemáticas y estabiliza el cálculo cuando el corpus es pequeño.

2.5 Peso TF-IDF

El peso final combina ambas ideas:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Un término recibe peso alto cuando cumple dos condiciones:

- Aparece con fuerza dentro del documento.
- No aparece de forma excesiva en todos los documentos.

Esto explica por qué `gasolina` puede ser importante, pero no siempre queda en primer lugar: aparece en varios documentos, así que su IDF baja. En cambio, términos como `filas`, `ductos` o `muro fronterizo` ayudan más a distinguir textos específicos.

2.6 Corpus usado en el ejemplo

El archivo `ejemplo/datos.csv` contiene diez documentos cortos. Fueron escritos con suficiente longitud para que el algoritmo pueda distinguir mejor frecuencia local y rareza global.

ID	Documento
1	El presidente anunció nuevas medidas para mejorar el abasto de gasolina... El plan de abasto incluye distribución de gasolina...
2	La falta de gasolina provocó largas filas... En varias estaciones las filas crecieron porque la gasolina llegó tarde...
3	Universidades públicas solicitaron mayor presupuesto... proyectos de investigación y programas de becas.

ID	Documento
6	Trump defendió la construcción del muro fronterizo... Trump insistió en que el muro fronterizo era necesario...
9	Autoridades investigan una red dedicada al robo de gasolina en ductos... venta ilegal de gasolina...

2.7 Preprocesamiento aplicado

Antes de calcular TF-IDF, el vectorizador realiza varias transformaciones.

Minúsculas

Todo el texto se normaliza con `lowercase=True`.

Stopwords

Se eliminan palabras como "el", "de", "para", "con".

Tokenización

Se extraen palabras con letras, incluyendo acentos y ñ.

N-gramas

Se usan palabras individuales y frases de dos palabras.

La lista de *stopwords* no cambia el significado central del corpus; su función es retirar términos gramaticales que aparecen mucho pero distinguen poco.

2.8 Implementación en el ejemplo

El script principal es `ejemplo/tfidf.py`. La parte esencial es la construcción de `TfidfVectorizer`.

► Mostrar código

2.8.1 Parámetros relevantes

Parámetro	Función técnica	Efecto en el ejemplo
<code>lowercase=True</code>	Normaliza mayúsculas y minúsculas	<code>Trump</code> y <code>trump</code> se tratan igual
<code>stop_words=STOPWORDS</code>	Elimina palabras poco informativas	Reduce ruido en la matriz
<code>token_pattern</code>	Define qué cuenta como token	Conserva palabras con acentos
<code>ngram_range=(1, 2)</code>	Usa unigramas y bigramas	Permite términos como <code>muro fronterizo</code>

Parámetro	Función técnica	Efecto en el ejemplo
<code>sublinear_tf=True</code>	Usa escala logarítmica para TF	Evita que una repetición domine demasiado
<code>smooth_idf=True</code>	Suaviza el IDF	Estabiliza el cálculo con pocos documentos
<code>norm="l2"</code>	Normaliza cada vector	Hace comparables documentos de distinta longitud

El uso de bigramas es importante: no solo aparecen palabras como `muro` o `fronterizo`, sino frases como `muro fronterizo`, que suelen ser más descriptivas.

2.9 Flujo del programa

El programa sigue una secuencia corta pero muy típica en minería de textos.

► [Mostrar código](#)

El primer archivo de salida, `matriz_tfidf.csv`, contiene toda la matriz documento-término. El segundo, `palabras_representativas.csv`, resume los cinco términos con mayor peso por documento.

2.10 Resultados principales

Los resultados muestran que los pesos ya no son idénticos para todas las palabras. Al ampliar los documentos, algunos términos se repiten dentro de un mismo texto y otros aparecen en más de un documento, lo que modifica TF e IDF.

Documento	Término	TF-IDF	Lectura
1	abasto	0.277280	El documento enfatiza la política de abasto
2	filas	0.282687	La consecuencia visible del desabasto domina el texto
3	investigación	0.199350	La demanda universitaria gira alrededor de investigación, presupuesto y becas
6	discurso	0.298094	El contexto discursivo se repite y diferencia el documento

Documento	Término	TF-IDF	Lectura
9	ductos	0.231510	El robo de gasolina se ubica en infraestructura específica

2.11 Interpretación técnica de tres documentos

2.11.1 Documento 2

El texto habla de falta de gasolina, filas y estaciones. El término con mayor peso es **filas**:

filas obtiene peso alto porque aparece más de una vez en el documento y no aparece como término dominante en los demás textos.

Término	TF-IDF
filas	0.282687
estaciones	0.210243
gasolina	0.186921
tarde conductores	0.166960
tarde	0.166960

Aunque **gasolina** es importante, aparece en otros documentos del corpus. Por eso su IDF es menor que el de términos más específicos.

2.11.2 Documento 3

El documento combina educación superior, presupuesto, investigación y becas.

Término	TF-IDF
investigación	0.199350
presupuesto	0.199350
becas	0.199350
sostiene laboratorios	0.178061
solicitaron mayor	0.178061

Los tres primeros términos tienen el mismo peso porque aparecen con frecuencia comparable y tienen una rareza global parecida dentro del corpus.

2.11.3 Documento 6

El texto se centra en Trump, un discurso y el muro fronterizo.

Término	TF-IDF
discurso	0.298094
muro fronterizo	0.298094
muro	0.298094
fronterizo	0.298094
trump	0.253407

Trump aparece también en el documento 7, por eso su IDF baja ligeramente. En cambio, **muro fronterizo** es más específico del documento 6 y conserva un peso mayor.

2.12 Qué produce exactamente el algoritmo

El resultado de TF-IDF no es una clasificación ni un resumen automático. Es una transformación:

texto crudo $\longrightarrow$ matriz numérica documento-término

Esa matriz puede usarse después para:

- buscar documentos similares;
- entrenar clasificadores;
- hacer agrupamiento;
- extraer palabras clave;
- comparar documentos por distancia coseno.

TF-IDF no entiende sinónimos ni contexto profundo. Si dos documentos usan palabras distintas para expresar la misma idea, TF-IDF puede tratarlos como diferentes. Por eso funciona bien como representación clásica e interpretable, pero no sustituye a modelos semánticos modernos.

2.13 Conclusión

TF-IDF es una técnica central en minería de textos porque convierte documentos en vectores interpretables. Su fuerza está en equilibrar frecuencia local y rareza global: una palabra importa si aparece dentro del documento, pero importa más si además ayuda a diferenciarlo del resto.

En este ejemplo, la implementación con **TfidfVectorizer** permite pasar de un CSV con noticias cortas a dos

salidas útiles: una matriz completa de pesos y una tabla con los términos más representativos de cada documento.

3 Modelado de tópicos (LDA)

Una de las preguntas más simples, pero más importantes a la hora de leer y extraer información de un texto es el determinar cuál o cuáles son los temas centrales en los que gira todo el texto, de manera mucho más coloquial: ¿de qué trata el texto que leí?

Puede ser una pregunta bastante trivial, pero es una consideración vital cuando estamos manejando una gran cantidad de textos, ya que surge la necesidad de agrupar grandes cantidades de datos en un número determinado de grupos, en este caso, que junte los textos que tienen mucho parecido entre sí. ¿Qué mejor que agrupar dichos textos en sus temas centrales?

El análisis de tópicos es un caso particular de los recursos vistos con anterioridad en este trabajo, en el que la frecuencia de patrones en los textos se usan para armar un conjunto (los temas centrales) y con este conjunto de datos vamos asignando un grupo por cada texto que compartimos con un modelo de agrupamiento. Una herramienta poderosa que nos ayudará a responder a la pregunta principal sobre el tema central de un texto y nos permitirá clasificar la información de una manera automática y eficiente.

3.0.1 ¿Qué es un tópico?

Se define a un tópico como el tema de una conversación o una discusión. En este caso del procesamiento de textos, cuando hablemos de tópicos nos referiremos al tema del que trata un texto.

4 Construir un predictor de tópicos

Haremos un ejemplo de un modelo que pueda analizar el tema central de un texto y pueda predecir los tópicos de cualquier texto que le lleguemos a pasar.

4.1 Consideraciones

Es necesario mencionar que la amplia mayoría de los textos no tienen un solo tópico, al ser un contenido completamente pedagógico, mantendremos el modelo y el escenario bastante simple, por lo que no consideramos esos casos, solo notamos su existencia.

También nos saltaremos el proceso de limpieza de datos, asumimos que los textos que recibimos no contienen un error en su formato o errores ortográficos ya que la fuente de información es un medio pequeño y controlado, además, se omite este proceso ya que se considera que no es relevante para el tema de análisis de tópicos

4.2 Escenario

Una página web de tutoriales quiere clasificar sus guías en 4 categorías: **Recetas de cocina**, **Papiroflexia**, **Alfarería** y **Carpintería**. Esta página cuenta con todos sus documentos en el mismo formato y sin ningún error ortográfico, ya que se lleva a cabo un proceso de revisión antes de la publicación de tutoriales.

Contamos con los textos en un CSV llamado `datos.csv`, que contiene los siguientes textos:

id_texto	texto
1	La receta de reposteria indica batir harina para el pastel. Mete el pastel al horno para terminar la receta.
2	Esta receta de chocolate lleva harina en el pastel. Calienta el pastel en el horno para tu receta.
3	El origami consiste en realizar un pliegue en el papel. Cada pliegue de papel genera una figura de origami.
4	Para el origami asiatico el pliegue de papel es vital. Arma una figura de papel usando el origami.
5	La ceramica artesanal usa arcilla en el torno. Tornea la arcilla para hacer una vasija de ceramica.
6	Trabajar la ceramica requiere poner arcilla sobre el torno. Gira el torno con arcilla y obtendras tu vasija de ceramica.
7	La carpinteria trabaja la madera del tablon. Corta el tablon de madera para construir el mueble de carpinteria.
8	En la carpinteria lijas la madera de cada tablon. Arma el mueble de madera en tu taller de carpinteria.

4.3 Preprocesado

Antes de enviar los textos a nuestro modelo de tópicos, debemos de asegurarnos que los documentos no contengan ningún caracter especial o elemento adicional que se entrometa en nuestro análisis. Estos objetos indeseables pueden parrafos o saltos de línea que no muestran alguna información relevante acerca de los tópicos por si mismos. además, procedemos a eliminar cualquier caracter especial, pues por si solos tampoco ofrecen ninguna información relevante acerca de los tópicos; también se hace una revisión adicional para eliminar cualquier otro error extraño, tal como dobles espacios o cualquier otro error que pudo haber sido hecho en el proceso de transferencia.

Después viene otro proceso especial conocido como la eliminación de **stopwords** o palabras vacias. Estas stopwords son aquellas palabras que se utilizan para dar más detalle o sentido a una idea al conjugarse con otras palabras, pero por sus propios medios no tienen sentido, es importante eliminarlas justo por ese motivo, ya que son palabras que por si solas no ayudan a determinar los tópicos de un texto y generan un ruido que puede confundir a nuestro modelo.

Todo el proceso en conjunto nos devolverá documentos de texto con palabras de significado puro y con las que ya podemos trabajar para construir un modelo.

```
def cargar_datos(ruta_csv):  
    """Carga el CSV y valida que existan las columnas esperadas."""
```

```

datos = pd.read_csv(ruta_csv)

columnas_necesarias = {"id", "texto"}
columnas_faltantes = columnas_necesarias - set(datos.columns)

if columnas_faltantes:
    raise ValueError(f"Faltan columnas en el CSV: {columnas_faltantes}")

datos["texto"] = datos["texto"].fillna("").astype(str)
return datos

def construir_vectorizador():
    """
    Configura CountVectorizer.

    LDA requiere una matriz de conteo de frecuencias (valores enteros)
    en lugar de una matriz con pesos continuos como TF-IDF.
    """
    return CountVectorizer(
        lowercase=True,
        stop_words=STOPWORDS,
        token_pattern=r"(?u)\b[a-záéíóúñü][a-záéíóúñü]+\b",
    )

```

Así tenemos estos datos después de la poda:

id_texto	texto_limpio
1	receta reposteria principal batir azucar harina pastel pastel dulce requiere hornear horno terminar receta
2	receta dulce chocolate debes mezclar azucar harina pastel prepara pastel dulce procede hornear horno receta
3	origami exige precision cada doblez papel realiza pliegue papel formar figura papiroflexia origami
4	papiroflexia cada doblez papel importante pliegue papel forma figura perfecta origami papiroflexia
5	alfareria artesanal debes moldear arcilla torno ceramica ayuda moldear arcilla formar vasija alfareria
6	moldear ceramica necesita paciencia torno arcilla usa alfareria moldear vasija ceramica
7	carpinteria necesitas madera pino tablon utiliza martillo madera armar mueble carpinteria

4.4 Frecuencia de terminos.

Ahora hacemos una matriz de frecuencia para recopilar todas las palabras puras de cada uno de los textos y el número de veces que se repiten en cada uno.

```
def calcular_dtm(datos):
    """Calcula la matriz de conteos (DTM) y regresa el DataFrame legible y los
    términos."""
    vectorizador = construir_vectorizador()
    matriz_conteos = vectorizador.fit_transform(datos["texto"])
    terminos = vectorizador.get_feature_names_out()

    # Convertir la DTM dispersa en un DataFrame estructurado
    df_dtm = pd.DataFrame(
        matriz_conteos.toarray(),
        columns=terminos,
        index=datos["id"]
    )
    df_dtm.index.name = "id"

    return matriz_conteos, df_dtm, terminos
```

4.5 Función LDA

La función LDA es el método que nos permite hacer la agrupación de textos desde 0. La función inicialmente recibe todas las palabras puras únicas que se recopilaron en todos los texto, además de recibir como parámetro una K que es la cantidad de categorías en las que queremos clasificar los textos. De manera inicial, LDA asigna “pesos” (la probabilidad de que si una palabra reside en un texto cualquiera, este pertenezca a la categoría “X”) aleatorios en las palabras

Pasa palabra por palabra, documento por documento, recalculando probabilidades. Se pregunta: ¿Qué tan común es este tópico en este tutorial? y ¿Qué tan común es esta palabra en este tópico?.

Repita este escaneo miles de veces hasta que las palabras se agrupan de forma lógica. En nuestro ejemplo: Las palabras de cocina se atraen entre sí, las de papiroflexia hacen lo mismo, y las categorías se vuelven nítidas.

Además, se puede configurar tanto el número de pasadas que se le darán a los textos para ajustar los pesos, una semilla que haga la asignación de pesos inicial a las palabras, y la forma de análisis de los textos, siendo **batch** un análisis completo de todo el lote de información, mientras que **online** lo hace por partes.

```
def ajustar_lda(matriz_conteos):
    """
    Instancia y entrena el modelo LDA con hiperparámetros estrictos
```

Instancia y entrena el modelo LDA con hiperparámetros optimizados

```
"""
k = 4
lda_model = LatentDirichletAllocation(
    n_components=k,
    doc_topic_prior=0.1,      # Alpha bajo: fuerza asignación a un único tópico
                              # dominante
    topic_word_prior=0.1,    # Beta bajo: reduce la distribución cruzada de
                              # palabras entre tópicos
    learning_method="batch",  # Evalúa todo el dataset de golpe
    max_iter=1000,           # Asegura iteraciones suficientes para estabilizar
                              # probabilidades
    random_state=42,         # Semilla fija
)

matriz_probabilidades = lda_model.fit_transform(matriz_conteos)
return lda_model, matriz_probabilidades
```

4.6 Evaluación

Ahora que tenemos a nuestro modelo entrenado, podemos transferirle nuestros documentos para que los coloqué en las categorías ya marcadas. De nuestro modelo hecho con LDA podemos extraer información relevante, por ejemplo, cuáles fueron las palabras más importantes para determinar si un texto pertenecía a un tópico.

```
def obtener_palabras_representativas(lda_model, terminos, top_n=5):
    """Obtiene las palabras con mayor peso asignado por el modelo para cada tópico."""
    resultados = []

    for id_topico, componente in enumerate(lda_model.components_, start=1):
        indices_ordenados = componente.argsort()[::-1]

        for ranking, indice_termino in enumerate(indices_ordenados[:top_n], start=1):
            resultados.append(
                {
                    "Tópico": f"Tópico {id_topico}",
                    "Ranking": ranking,
                    "Termino": terminos[indice_termino],
                    "Peso Psicológico": round(float(componente[indice_termino]), 4),
                }
            )

    return pd.DataFrame(resultados)
```

Ahora, pasamos nuestros textos por el modelo para ver la probabilidad de que pertenezcan a cada tópico.

```
def crear_matriz_probabilidades(datos, matriz_probabilidades):
    """Convierte la matriz de distribución documento-tópico en una tabla legible."""
    k = matriz_probabilidades.shape[1]
    columnas_topicos = [f"Tópico {i+1}" for i in range(k)]

    matriz = pd.DataFrame(
```

```

matriz_probabilidades,
columns=columnas_topicos,
index=datos["id"],
)
matriz.index.name = "id"
return matriz.round(4)

```

De manera simplificada, estos fueron los resultados que obtuvimos:

id_texto	texto	tópico
1	La receta de reposteria indica batir harina para el pastel. Mete el pastel al horno para terminar la receta.	Tópico 3 (Recetas de cocina)
2	Esta receta de chocolate lleva harina en el pastel. Calienta el pastel en el horno para tu receta.	Tópico 3 (Recetas de cocina)
3	El origami consiste en realizar un pliegue en el papel. Cada pliegue de papel genera una figura de origami.	Tópico 4 (Papiroflexia)
4	Para el origami asiatico el pliegue de papel es vital. Arma una figura de papel usando el origami.	Tópico 4 (Papiroflexia)
5	La ceramica artesanal usa arcilla en el torno. Tornea la arcilla para hacer una vasija de ceramica.	Tópico 1 (Alfarería)
6	Trabajar la ceramica requiere poner arcilla sobre el torno. Gira el torno con arcilla y obtendras tu vasija de ceramica.	Tópico 1 (Alfarería)
7	La carpinteria trabaja la madera del tablon. Corta el tablon de madera para construir el mueble de carpinteria.	Tópico 2 (Carpintería)
8	En la carpinteria lijas la madera de cada tablon. Arma el mueble de madera en tu taller de carpinteria.	Tópico 2 (Carpintería)

4.7 Consideraciones finales

Nunca es buena idea entrenar modelos con poca información: Pocos datos implican mayor incertidumbre para el modelo.

Pocas palabras que coincidan pueden confundir al modelo, empeora si son pocas entradas

Se puede observar en la siguiente gráfica lo certero que estaba el modelo a la hora de asignar un tópico

Nuestro modelo es muy pequeño, así que cualquier palabra que compartan entre ellos juega un papel muy importante para determinar a qué categoría pertenece. Se recalca la importancia de **Stopwords** ya que hay muchas palabras que comparten todos los textos, que por si mismas no son útiles para el análisis, pero por la construcción del español son usadas para generar enunciados; así que tener un mal “podado” de datos puede suponer una mayor confusión por parte de nuestro modelo.

También puede ocurrir que simplemente después del entrenamiento, nuestro modelo no esté completamente seguro de los tópicos a los que pertenece un texto, en nuestro dataset de ejemplo podemos observar un pequeño rastro de este fenómeno al obtener las probabilidades de que un texto pertenezca un tópico

Distribución de Probabilidad Documento-Tópico

ID del Documento	Tópico 1	Tópico 2	Tópico 3	Tópico 4
1	0.0088	0.0088	0.9737	0.0088
2	0.0065	0.0065	0.9805	0.0065
3	0.0081	0.0081	0.0081	0.9758
4	0.0088	0.0088	0.2719	0.7106
5	0.7339	0.0081	0.2500	0.0081
6	0.9712	0.0096	0.0096	0.0096
7	0.0088	0.9737	0.0088	0.0088
8	0.0096	0.9712	0.0096	0.0096
	Tópico 1	Tópico 2	Tópico 3	Tópico 4

Tópicos Latentes

Entre el texto 4 y 5, el modelo estaba 70% seguro (aprox) de que los textos pertenecían a su tópico asignado. Por compartir algunas palabras, el modelo estaba 20% seguro (aprox) de que pertenecían a otro tópico.

Para ver más de este fenómeno de inseguridad, puede ejecutarse el script de python sobre nuestro dataset alternativo `data_alt`, ahí se puede observar a un modelo que no está completamente seguro de a qué tópico pertenecen los textos.

5 Análisis de sentimientos

[Steve]

